Submitted by:
M.Latif Tariq , 2026(S)-CYS-42

Section "A"

Submitted to:
Sir Hafiz Mubashar

ALL LAB TASKS AND ASSIGNMENTS

For the fulfillment of,

Programming Fundamental Lab

Department of Computer Engineering

University of Engineering and Technology

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: Manuals Report	**Dated: 29/6/2026**
Semester: 1	**Session: Spring-2026**
Lab/Project/Assignment #: 1	**CLOs to be covered : CLO-1**
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else), For Loops, While loop, Nested loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

CLO-1

LAB No 1

Lab Goals/Objectives:

Basics & Logic: Gain hands-on experience in handling user input/output, selecting appropriate data types, and utilizing operators for precise calculations. **Control Flow:** Master structural logic by implementing if/else decisions and optimizing code efficiency using for, while, and nested loops to solve complex, repetitive computational problems.

Generating Output using user defined and built-in functions. Finding Data Types of variables and displaying your personal info. So that you can understand the concepts of I/O and data types.

Equipment Required:

Machine: Laptop or PC.
P-Language: python-3.14
IDE: Visual Studio
Some built-in functions like (print, if-else, elif) and some user defined functions, plus variables.

Theory:

- **Input/Output:** Input takes user data (via keyboards/sensors) into a program, while Output displays processed results (via screens/speakers) to the user.
- **Data Types:** Classifications (like integers, floats, strings, or booleans) that tell the computer how to store and manipulate specific data values.
- **Operators:** Special symbols (such as `+`, `-`, `==`, `&&`) used to perform mathematical, relational, or logical calculations on data values.
- **Conditional Statements ($If/Else$):** Decision-making structures that execute specific blocks of code only if a defined condition evaluates to true or false.
- **For Loop:** A control flow statement used to repeat a specific block of code a predetermined number of times, typically utilizing a counter or sequence.
- **While Loop:** A condition-driven loop that continuously executes a block of code as long as a specified boolean condition remains true.
- **Nested Loop:** A loop placed inside the body of another loop, where the inner loop completes all its iterations for every single iteration of the outer loop.

Lab Work:

Task 1:

```
name = input("Enter your name:")
age = int(input("Enter your age:"))
print("Name:",name)
print("Your age:",age)
```

Result:

```
PS D:\Vs code\Microsoft VS Code> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/1st prog.py"
Enter your name:Latif
Enter your age:19
Name: Latif
Your age: 19
PS D:\Vs code\Microsoft VS Code>
```

Task 2:

```
Name: Latif
Age: 19
City: Lahore
print("Name:Latif\nAge:19\nCity:Lahore")
```

Result:

```
PS D:\Vs code\Microsoft VS Code>

                              > & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/1st prog.py"
Name:Latif
Age:19
City:Lahore
PS D:\Vs code\Microsoft VS Code>
```

Task 3:

```
if s=='a'or s=='e'or s=='i'or s=='o'or s=='u'or s=='A'or s=='E'or s=='I'or s=='O'or
s=='U':
        vowels+=1
    elif s!=" ":
        consonants+=1
```

Result:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

 *  Executing task: C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe 'd:\Python codes\Lab 2.py'

Sentence is: {'Hi its me'}
Total vowels are: {3}
Total consonants are: {4}
 *  Terminal will be reused by tasks, press any key to close it.
```

Task 4:

```
if temp == "":
    print("Atleast choose something!")
else:
    for i in range(1, length+1):
        password += random.choice(temp)

    print(f"Your Password: {password}")
```

Result:

```
PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab 2.py"
Enter password length: 14
Do you want to add Upper case Letters (yes/no): yes
Do you want to add Lower case Letters (yes/no): yes
Do you want to add Digits (yes/no): yes
Do you want to add Special Characters (yes/no): no
Your Password: s8CzX5L8dmXxpP
```

Conclusions:

These all above attached tasks contains all the python codes related to the CLO-1. Which shows that user has the clear understanding of all topics related to Input/Output, Data Types, Operators, Conditional Statements (If/Else).

LAB no. 2
For loop

Task 1:

```
for n in range(initial,final+1):
    count = 0
    for j in  range(1,n+1):
     if n % j == 0:
       count += 1
```

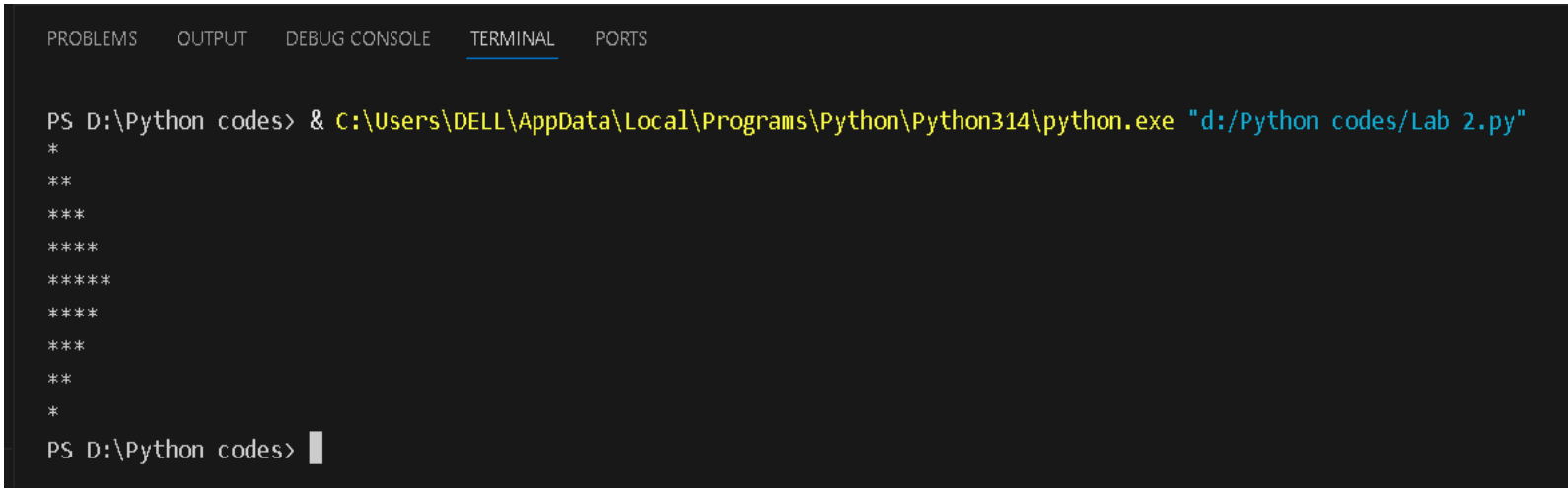
Result:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab 2.py"
Enter your range:2
Enter ur last value:10
  2
  3
  5
  7
sum of prime numbers 17
```

Task 2:

```
for i in range(1,6):
    for j in range(1,i+1):
        print("*",end="")
    print()
```

Results:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab 2.py"
*
**
***
****
*****
****
***
**
*
PS D:\Python codes>

Conclusion:

The implementation of For loops demonstrated their efficiency in handling definite iteration. By iterating over fixed sequences or ranges, they optimize code readability and prevent infinite loops, making them essential for predictable, counter-driven repetitive tasks.

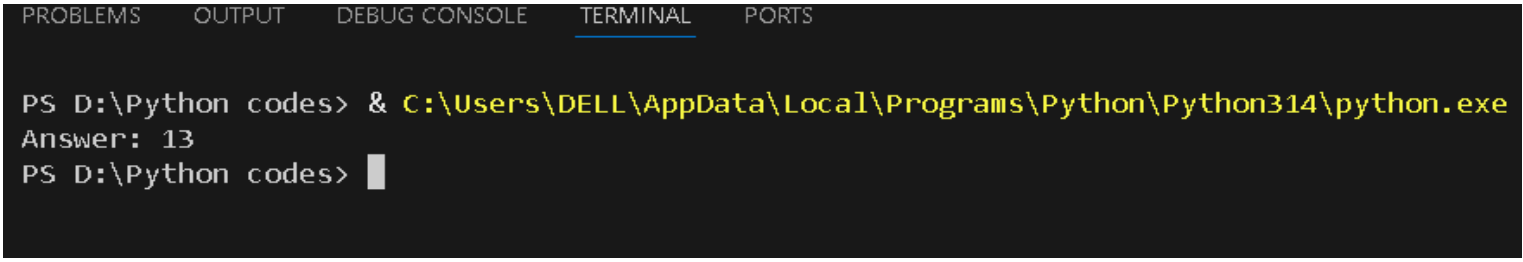
Lab no.3

While loop

Task 1:

```
while n != 0:
    sp_num = n % 10   # 1101 % 10 = 1 -->  sp_num = 1
    ans += sp_num * (2 ** i)
    n = n // 10
    i += 1

print(f"Answer: {ans}")
```

Result:



Conclusion:

The execution of while loops highlighted their utility in indefinite iteration, where the number of repetitions is unknown beforehand. They are vital for driving programs that must run continuously until a specific runtime condition or state changes.

Lab no.4
Nested loops

Task 1:

```
for i in range(1,6):
    for j in range(1,i+1):
        print("*",end="")
    print()
#Deals with lower '*'
for i in range(1,5):
    for j in range(5,i,-1):
        print("*",end="")
    print()
```

Result:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab 2.py"
*
**
***
****
*****
****
***
**
*
PS D:\Python codes>
```

Task 2:

```
for n in range(initial,final+1):
    count = 0
    for j in  range(1,n+1):
     if n % j == 0:
       count += 1

    if count == 2:
       print(" ",n)
       Sum_prime += n
print("sum of prime numbers",Sum_prime)
```

Result:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab 2.py"
Enter your range:2
Enter ur last value:10
  2
  3
  5
  7
sum of prime numbers 17
```

Conclusion:

Nested loops effectively demonstrated how to handle complex, multi-dimensional data and repetitive patterns. By executing an inner loop fully for every outer loop iteration, they proved crucial for managing matrices, grids, and multi-layered problem-solving.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: Manuals Report	**Dated: 29/6/2026**
Semester: 1	**Session: Spring-2026**
Lab/Project/Assignment #: 2	**CLOs to be covered : CLO-2**
Lab Title: Functions, Global and Local variables, Recursion, Lambda functions, MAP, Filter and Reduce, OS Library, if __name__ == "__main__", is vs ==	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.

Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

CLO-2

Lab Goals/Objectives:

This module covers advanced Python control flow, memory scope, and functional programming. The objective is to master code modularity using functions, variable scopes, and recursion, while optimizing data transformation via lambda, `map`, `filter`, and `reduce`. Additionally, labs focus on system-level automation using the `os` library, structural execution tracking via `__name__ == "__main__"`, and distinguishing object identity from equality.

Equipment Required:

Machine: Laptop or PC.
P-Language: python-3.14
IDE: Visual Studio

Theory:

- **Functions (Global & Local Variables):**
- Modular blocks for reusability. **Local variables** exist only inside the function, while **global variables** are accessible throughout the entire script, determining memory visibility.
- **Recursion:**
- A programmatic technique where a function calls itself to solve smaller sub-problems. It requires a **base case** to stop execution and prevent stack overflow.
- **Lambda Functions:**
- Small, anonymous, single-line functions defined using the `lambda` keyword. They execute a single expression and are used for short-lived, clean utility tasks.
- **Map, Filter, and Reduce:**
- Functional tools for iterables: `map()` transforms every element; `filter()` extracts items matching a condition; `reduce()` collapses the data down to a single value.
- **OS Library:**
- A built-in module allowing Python to interact directly with the operating system to automate file handling, folder creation, directory navigation, and system tasks.
- **`if __name__ == "__main__":`**
- A control idiom ensuring a code block runs only when the script is executed directly, preventing it from running when imported as a module.
- **`is` vs `==`:**
- The `==` operator evaluates structural value equality, while `is` checks memory identity, verifying if two variables reference the exact same object in RAM.

Lab Work
Lab no.5

Task 1:

```
def Comb(n, r):
    return fact(n) // (fact(r) * fact(n - r))
```

Result:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab_3.py"
Enter n: 4
Enter r: 3

P(4, 3) = 24
C(4, 3) = 4
PS D:\Python codes>
```

Task 2:

```
message = "I am global"

def my_function():
    text = "I am local"
    print(text)
    print(message)
my_function()
```

Result:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Loc
PF lab Week 1/Task 1.py"
I am local
I am global
PS D:\Python codes>
```

Conclusion:

Functions (Global & Local Variables): Demonstrated the importance of encapsulation and scope. Restricting variables to local scopes prevents accidental data corruption, while functions ensure code reusability and modular design.

Lab no.6
Recursion

Task 1:

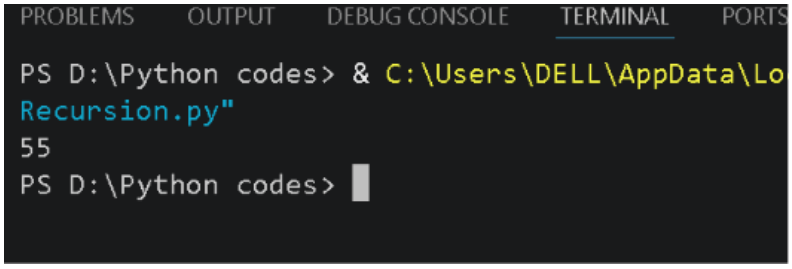
```
def show(n):
    if n == -1:
        return
    print(n)
    show(n-1)
show(5)
```

Result:

Task 2:

```
def sum(n):
    if n == 0:
        return 0
    return sum(n-1) + n

print(sum(10))
```

Result:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS D:\Python codes> & C:\Users\DELL\AppData\Lo
Recursion.py"
55
PS D:\Python codes>

Conclusion:

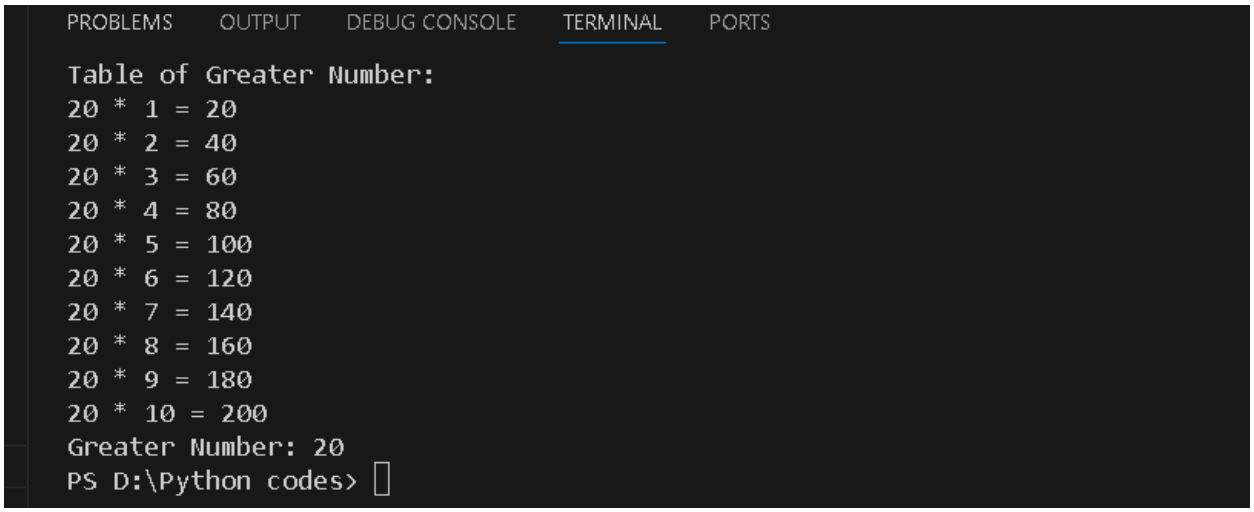
Recursion: Highlighted how complex problems can be elegantly solved by breaking them into smaller, self-referential sub-problems, emphasizing the critical role of base cases to prevent stack overflow.

Lab no. 7
Lambda Functions

Task 1:

```
greater = lambda num_1, num_2: num_1 if num_1 > num_2 else num_2

# Table Function of Greatest Number
def table(g):
    print("\nTable of Greater Number: ")
    for i in range(1, 11):
        print(f"{g} * {i} = {g*i}")
```

Result:



Task 2:

```
sent=input("Enter a string:")

upper_func = lambda a : a.upper()

upper_sent = upper_func(sent)
```

Result:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Python codes> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe "d:/Python codes/Lab_3.py"
Enter a string:yumiru
URIMUY
PS D:\Python codes>
```

Conclusions:

Lambda Functions: Proved highly efficient for creating quick, anonymous, single-line functions, drastically reducing boilerplate code for short-lived utility operations.

Lab no.8
MAP, Filter and Reduce

Task 1:

```
numbers = [1, 3, 2, 5]
square = list(map(lambda x : x**2, numbers))
print(square)
```

Result:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Python codes\AICT week 13> & C:\Users\DELL\AppDa
Python codes/AICT week 13/Map.py"
[1, 9, 4, 25]
PS D:\Python codes\AICT week 13>
```

Task 2:

```
numbers = [5, 2, 6, 3]
add = list(map(lambda x : x + 10, numbers))
print(add)
```

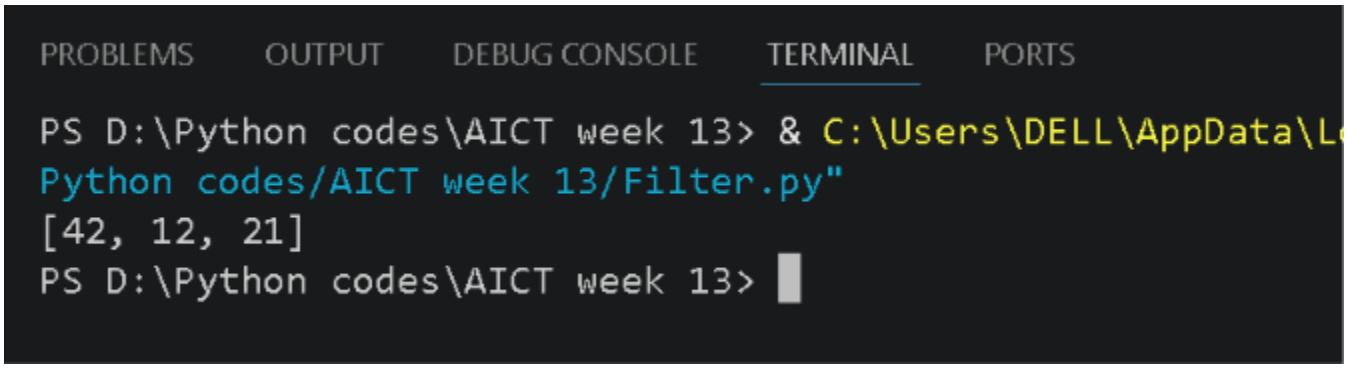
Result:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Python codes\AICT week 13> & C:\Users\DELL\AppData\Loca
Python codes/AICT week 13/Map.py"
[15, 12, 16, 13]
PS D:\Python codes\AICT week 13>
```

Task 3:

```
n = [10, 42, 12, 21]
larger = list(filter(lambda x :x > 10, n ))
print(larger)
```

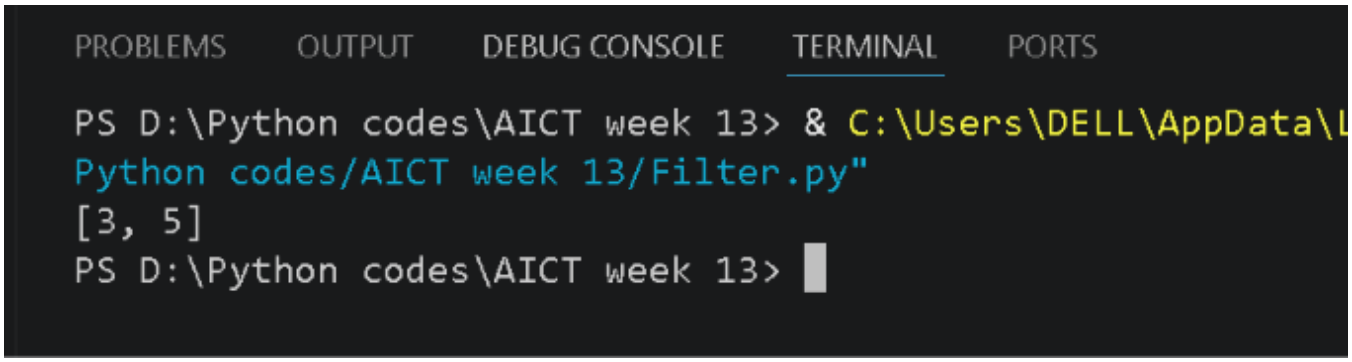
Result:



Task 4:

```
numbers = [3, 5, 6, 8]
odd = list(filter(lambda x : x % 2 != 0, numbers))
print(odd)
```

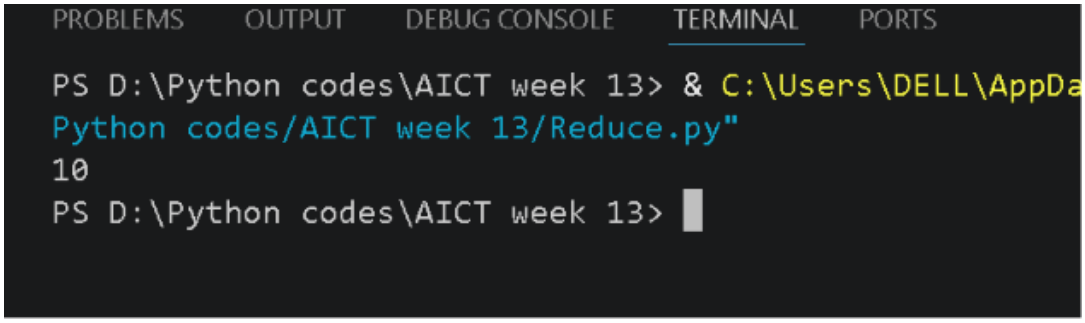
Result:



Task 5:

```
numbers = [1, 4, 3, 2]
add = reduce(lambda a, b : a + b, numbers)
```

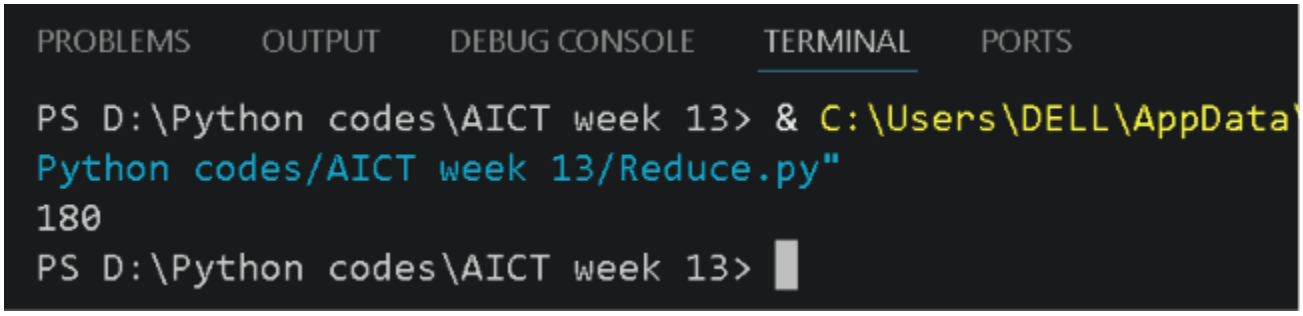
Result:



Task 6:

```
numbers = [3, 5, 6, 2]
mul = reduce(lambda a, b : a * b, numbers)
```

Result:



Conclusion:

Map, Filter, and Reduce: Combining these functional tools streamlined data manipulation by enabling elegant, mathematical, and highly efficient transformations over iterables without relying on explicit loops.

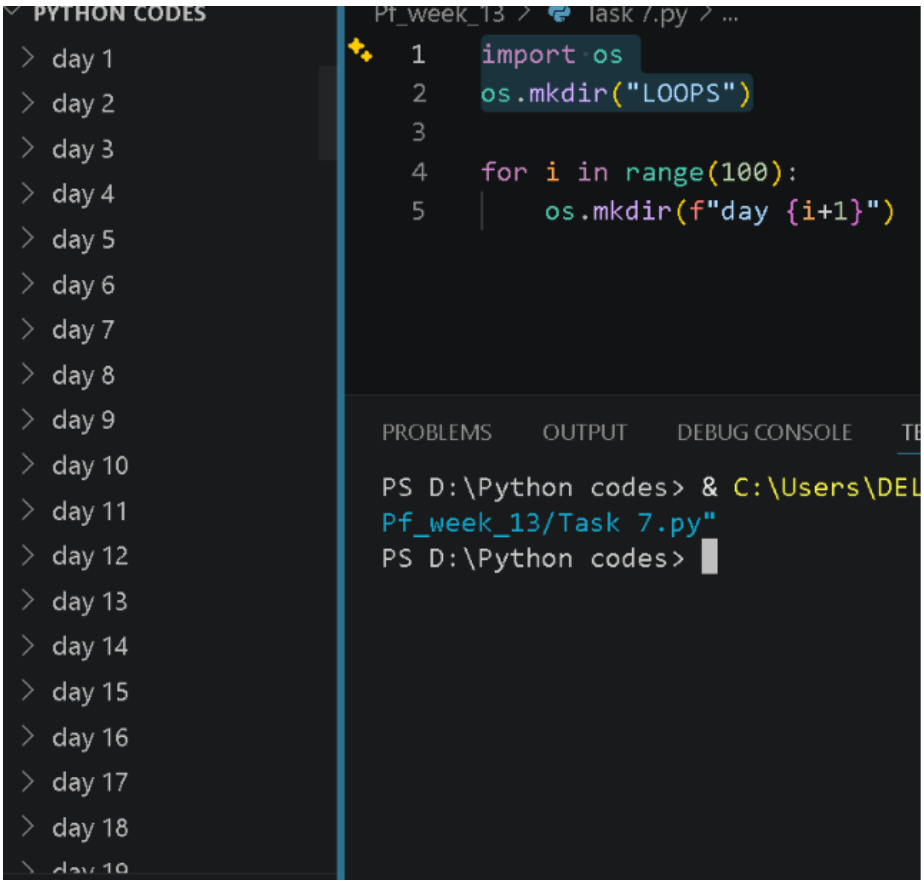
Lab no.9

OS Library, if __name__ == "__main__", is vs ==

Task 1:

```
import os
os.mkdir("LOOPS")
```

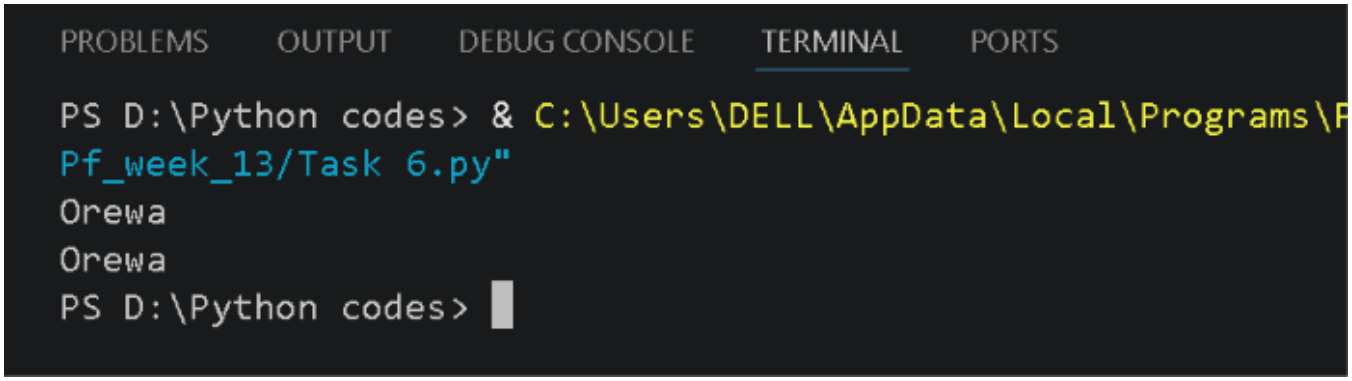
Result:


PYTHON CODES
day 1
day 2
day 3
day 4
day 5
day 6
day 7
day 8
day 9
day 10
day 11
day 12
day 13
day 14
day 15
day 16
day 17
day 18
Pf_week_13 > Task 7.py > ...
1 import os
2 os.mkdir("LOOPS")
3
4 for i in range(100):
5 os.mkdir(f"day {i+1}")
PROBLEMS
OUTPUT
DEBUG CONSOLE
PS D:\Python codes> & C:\Users\DEL
Pf_week_13/Task 7.py"
PS D:\Python codes>

Task 2:

```
import latif
if __name__ == "__main__":
    latif.welcome()
```

Result:



Task 3:

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)
```

Result:

Task 4:

```
a = [1, 2, 3]
b = a

print(a is b)
```

Result:

Conclusions: • **OS Library:** Demonstrated Python's capability to interact directly with the host operating system, proving essential for automating file management, directory navigation, and system-level scripting.

• **`if __name__ == "__main__":`** Showcased how to control script execution behavior, allowing files to safely serve dual purposes as both reusable modules and standalone executable scripts.

• **`is` vs `==`:** Clarified the critical distinction in memory management between structural value equality (`==`) and object reference identity (`is`), ensuring accurate logical comparisons.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: Manuals Report	**Dated: 29/6/2026**
Semester: 1	**Session: Spring-2026**
Lab/Project/Assignment #: 3	**CLOs to be covered : CLO-3**
Lab Title: List, Tuple, Sets, Dictionary, GUI using Qt Designer	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

CLO-3
Lab no.10

Lab Goals/Objectives:

- **Lists & Tuples (Sequence Management):** Learn to implement dynamic data storage using mutable **Lists** for ordered, changing data, and immutable **Tuples** for fixed, read-only sequences to ensure data integrity.
- **Sets & Dictionaries (Optimized Mapping):** Master fast lookups and data deduplication using **Sets** for unique elements, and **Dictionaries** for structured key-value mapping to handle complex datasets efficiently.
- **GUI Development (Qt Designer):** Transition from CLI to desktop application development by visually designing interactive, professional interfaces using **Qt Designer** and binding frontend widgets to backend Python logic.

Equipments Required:

Machine: Laptop or PC.
P-Language: python-3.14
IDE: Visual Studio
GUI: Qt designer

Theory:

- **List & Tuple: Lists** are mutable, ordered sequences for dynamic data manipulation. **Tuples** are immutable, fixed-size sequences that protect data integrity and optimize memory.

- **Sets & Dictionary: Sets** are unordered collections of unique elements for fast deduplication. **Dictionaries** store key-value pairs, using hashing for near-instant data retrieval.

- **GUI using Qt Designer:** A drag-and-drop visual layout tool used to design interactive desktop interfaces, cleanly separating frontend widgets from backend Python logic.

Lab no.10
List & Tuples

Task 1:

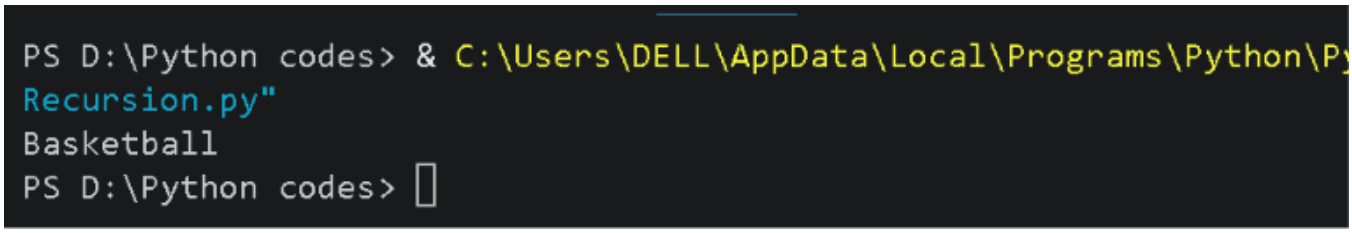
```
x = [1,2,3,4,5]

print(x[1:3])
print(x[:3])
```

Result:

Task 2:

```
sports = ('Badminton', 'Basketball', 'Football')
print(sports[1])
```

Result:



Conclusions: **List & Tuple:** Successfully balanced mutability and data safety, using lists for changing datasets and tuples to guarantee permanent, tamper-proof storage.

Lab no.11
Sets & Dictionary

Task 1:

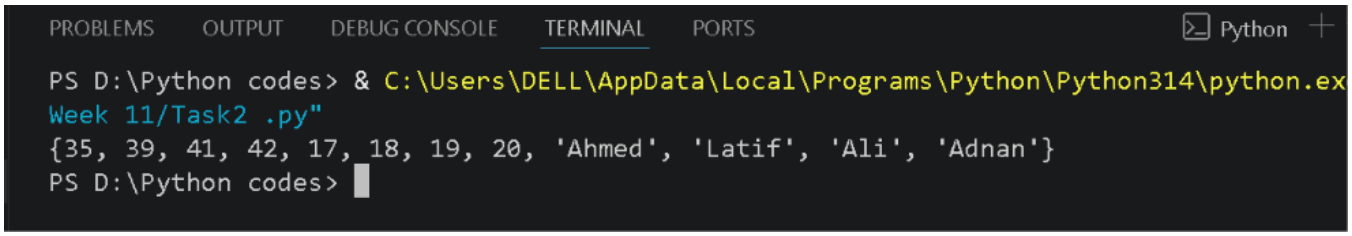
```
fruits  = {"apple": "saib", "banana": "kela", "orange": "malta", "grapes": "angoor",
"watermelon": "tarbooz", "pineapple": "ananas", "strawberry": "strawberry",
"blueberry": "blueberry", "kiwi": "kiwi", "mango": "aam"}
print("",fruits)
```

Results:

Task 2:

```
S1 = {'Latif', 'Adnan', 'Ali', 'Ahmed'}
S2 = {42, 39, 35, 41}
S3 = {19, 18, 17, 20}

a = set.union(S1, S2, S3)
print(a)
```

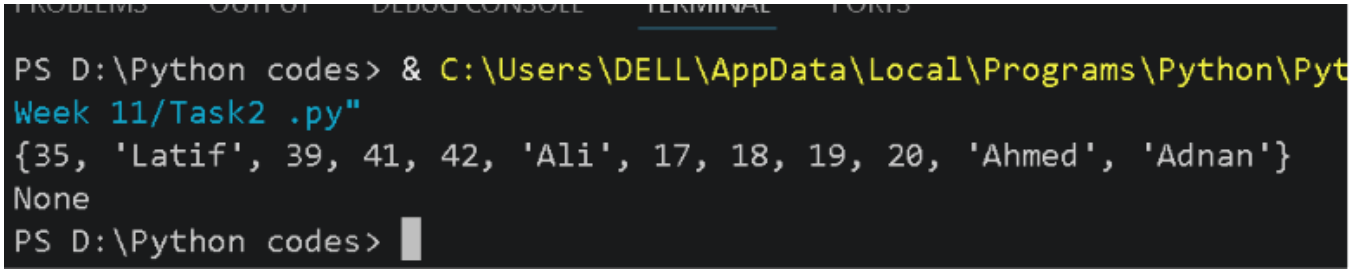
Result:



Task 3:

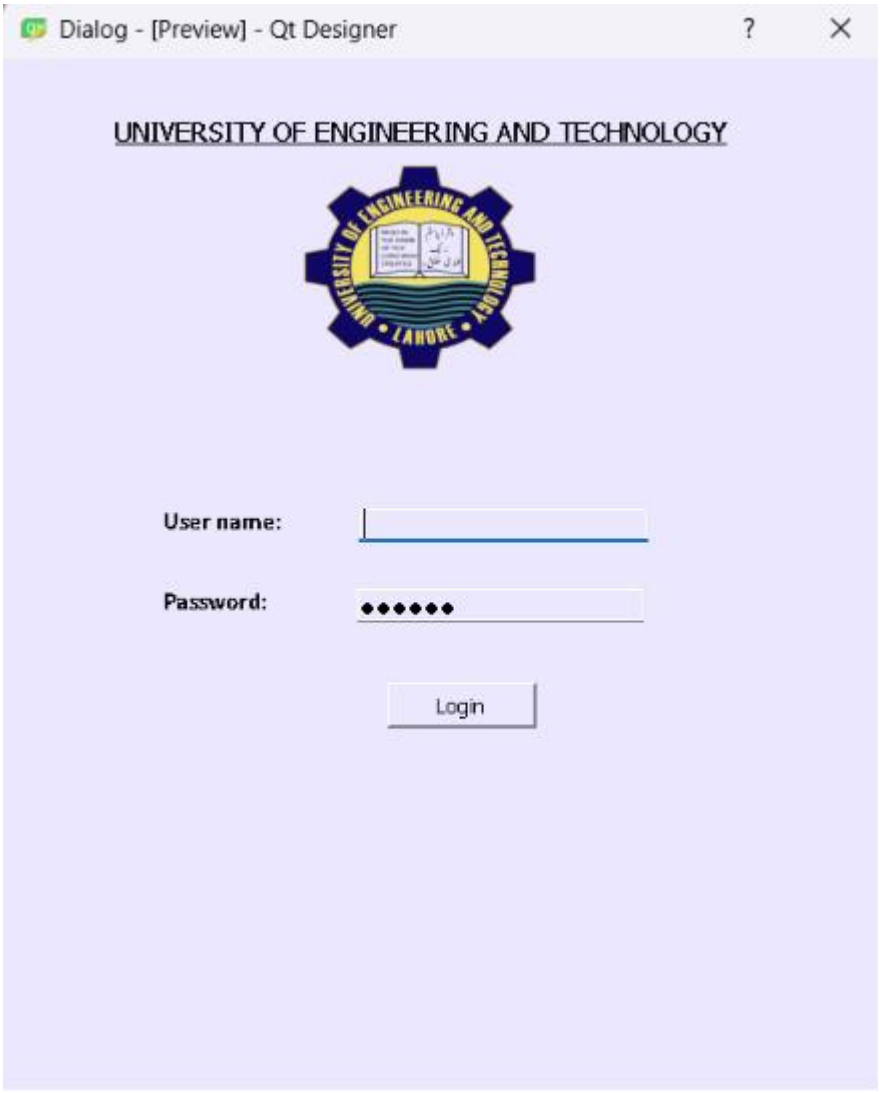
```
a = print(set.union(S1, S2, S3))
print(a)
```

Result:



Conclusions: **Sets & Dictionary:** Highlighted the power of hashing, where sets effectively eliminated duplicate entries and dictionaries provided ultra-fast, structured data mapping.

Lab no.12
GUI using Qt designer



Conclusions: **GUI using Qt Designer:** Successfully bridged the gap between code and user experience, creating a functional, professional desktop application from scratch.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: Manuals Report	**Dated: 29/6/2026**
Semester: 1	**Session: Spring-2026**
Lab/Project/Assignment #: 4	**CLOs to be covered : CLO-1**
Lab Title: Exception Handling	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

CLO-4

Lab Objectives/roles:

Master defensive programming by anticipating potential execution failures, validating user inputs, and implementing robust error-trapping routines to create crash-resistant code.

Equipments Required:

Machine: Laptop or PC.
P-Language: python-3.14
IDE: Visual Studio

Theory:

A mechanism that intercepts and manages runtime errors (like `ZeroDivisionError`), preventing abrupt crashes using `try`, `except`, `finally`, and `else` blocks to maintain smooth control flow.

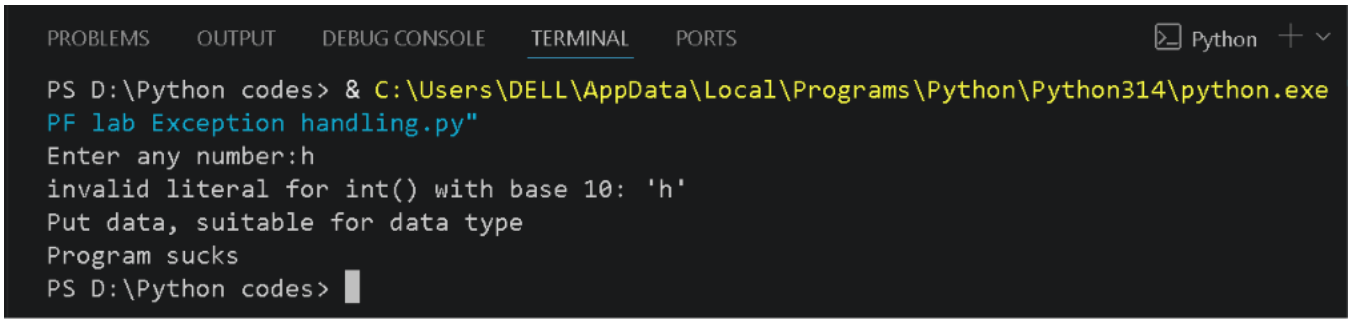
Lab no.13

"Exception Handling"

Task 1:

```
try:
    a = int(input("Enter any number:"))
    print(a)
except Exception as e:
    print(e)
    print("Put data, suitable for data type")
    print("Program sucks")
```

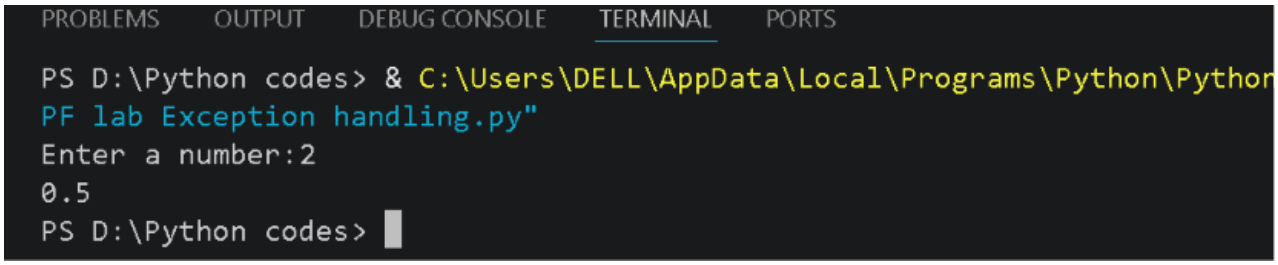
Result:



Task 2:

```
try:
    num = int(input("Enter a number:"))
    print(1/num)

except ValueError:
    print("put the correct values")
```

Result:



- **Conclusion:** Exception handling successfully shifted programs from failing unpredictably to failing gracefully. It proved indispensable for maintaining application uptime and securing software against unexpected runtime states.